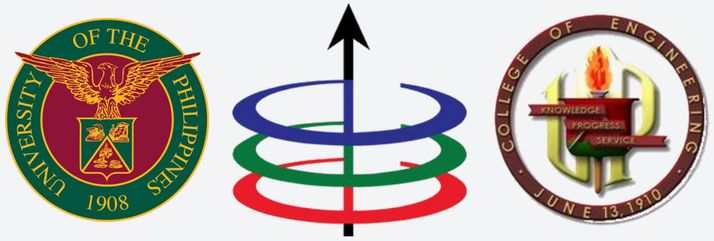


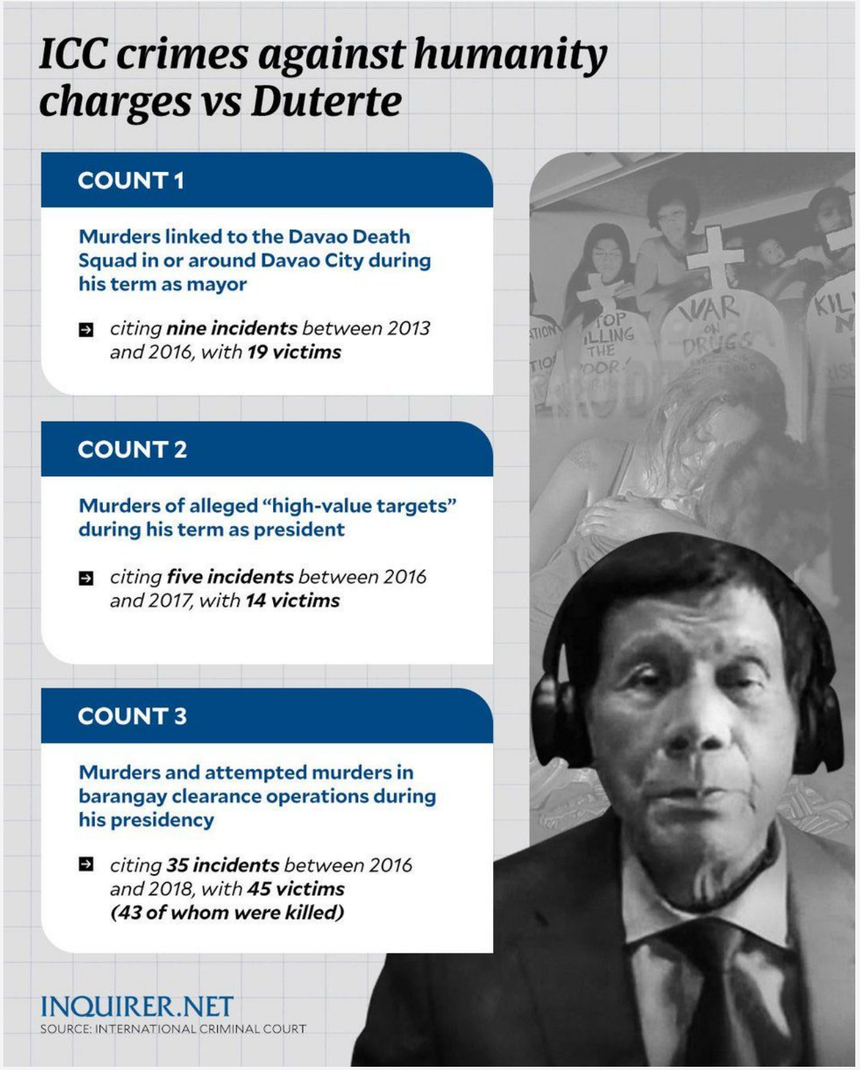
Lecture 4
Linear Data Structures

EEE 121 2s2526

Steven S. Sison


BREAKING
RODRIGO DUTERTE
ARRESTED


BREAKING
RODRIGO DUTERTE
ARRESTED


ICC crimes against humanity charges vs Duterte
COUNT 1
Murders linked to the Davao Death Squad in or around Davao City during his term as mayor
citing nine incidents between 2013 and 2016, with 19 victims
COUNT 2
Murders of alleged "high-value targets" during his term as president
citing five incidents between 2016 and 2017, with 14 victims
COUNT 3
Murders and attempted murders in barangay clearance operations during his presidency
citing 35 incidents between 2016 and 2018, with 45 victims (43 of whom were killed)
INQUIRER.NET
SOURCE: INTERNATIONAL CRIMINAL COURT
WAR ON DRUGS

- Data Structures: An Overview
- Arrays and Linked Lists
- Stacks and Queues

- **Data Structures: An Overview**
- Arrays and Linked Lists
- Stacks and Queues

Remember our primary goal, efficiency

Goal: Create efficient programs by using the most appropriate algorithm and data structures

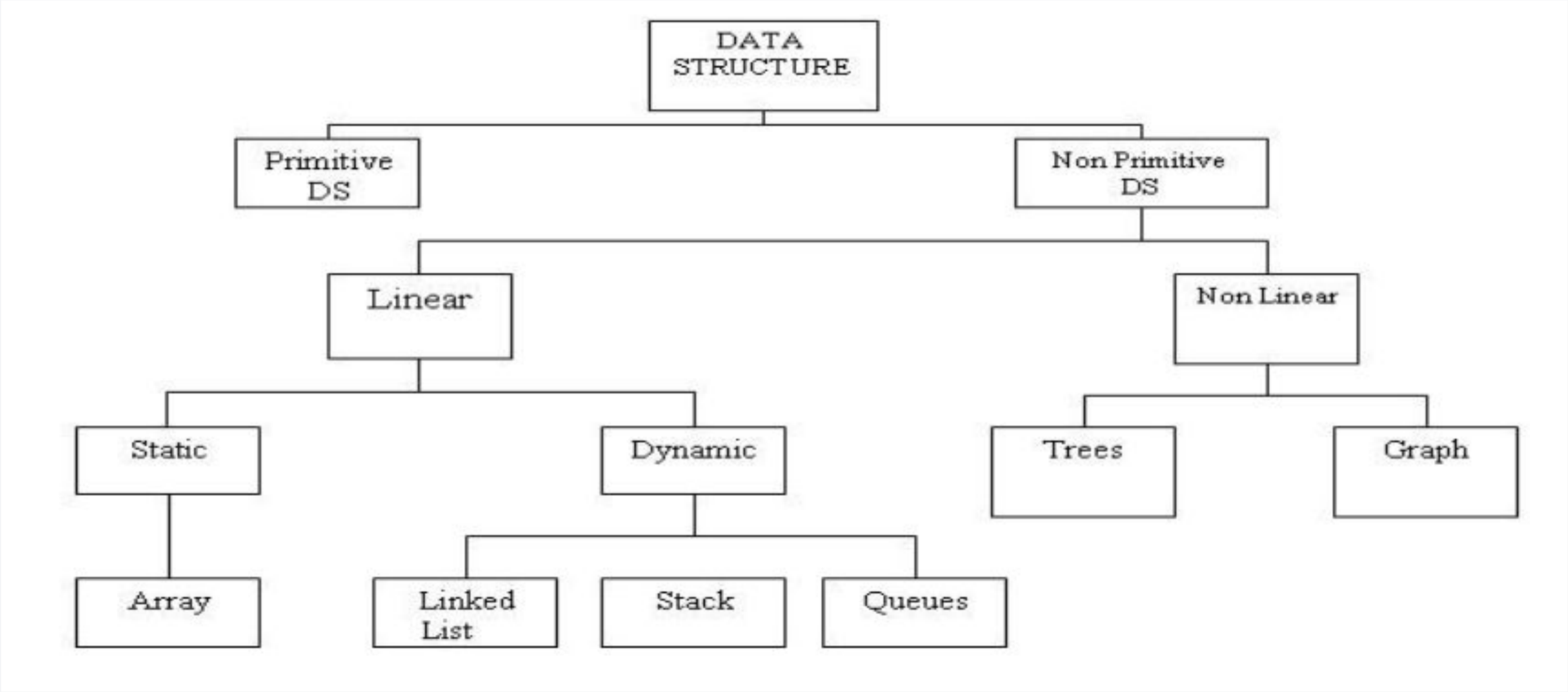
Remember our primary goal, efficiency

Goal: Create efficient programs by using the most appropriate algorithm and data structures

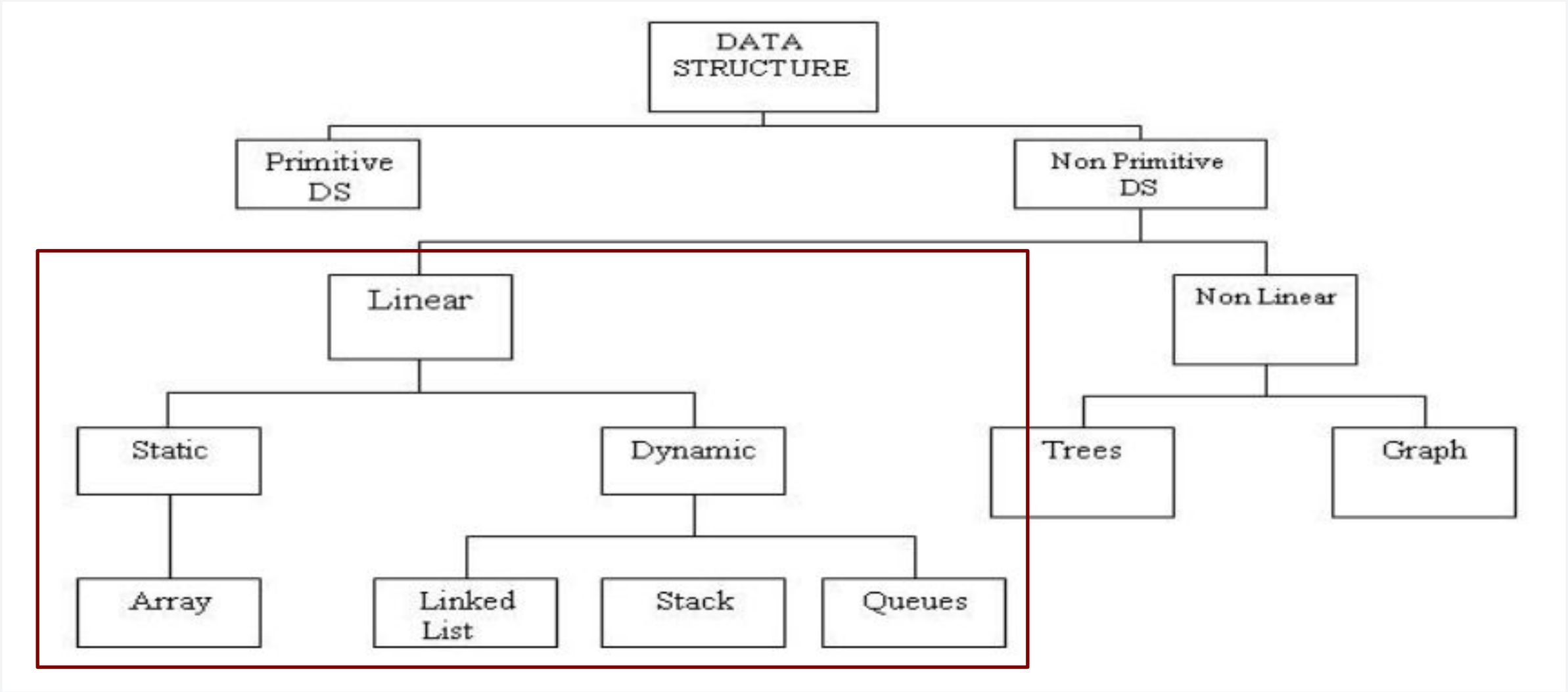
Data Structures

- A way of organizing and storing data in a program
- There is no perfect data structure, but there is a best data structure for a certain problem
  - Example:
    - Arrays for Stacks, Queues
    - Trees for Search

There are A LOT of data structures



There are A LOT of data structures



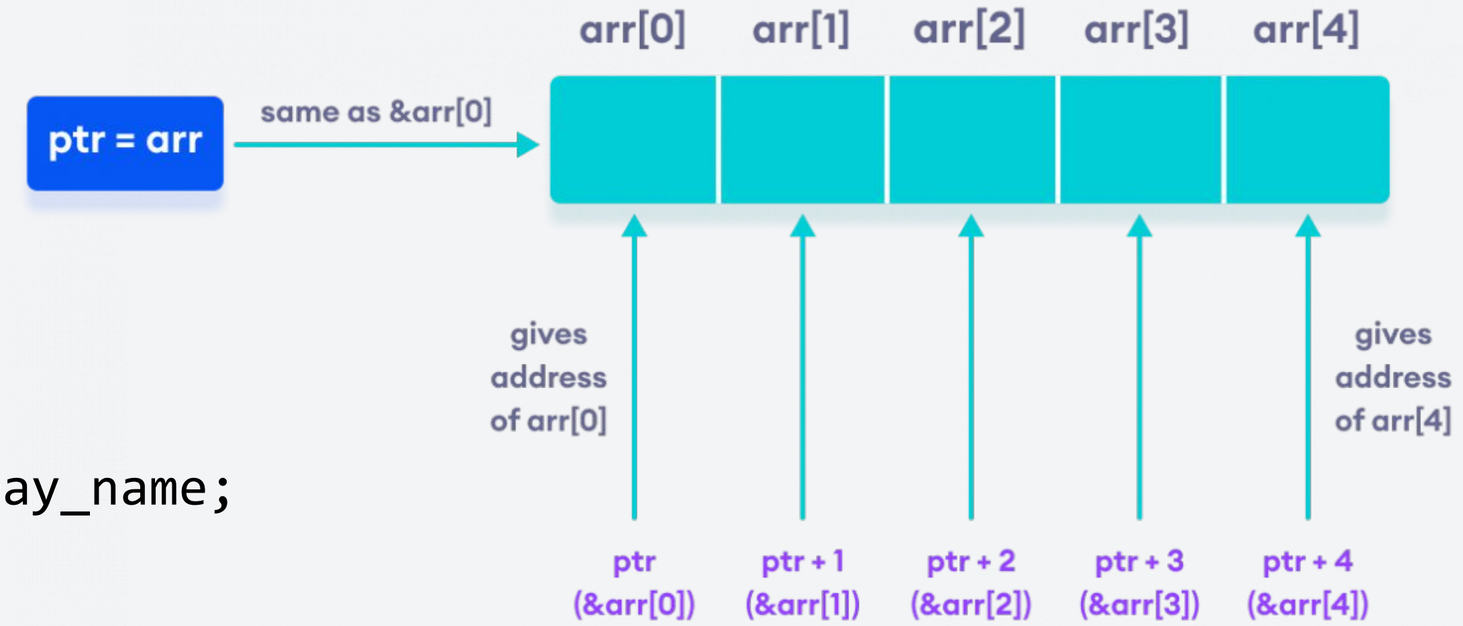
- Data Structures: An Overview
- **Arrays and Linked Lists**
- Stacks and Queues

LDS 1: Arrays

An array is essentially a collection of indexed elements having the same datatype.

- Physical placement in memory depends on the relative position in the array.

  Recall: arrays as pointers.



Initialization:

```
type array_name[size];
std::array<type,size> array_name;
```

There are two types of arrays...

Static Arrays

- static == fixed
- Memory is allocated at compile time
- Placed in the stack memory.

```
type array_name[size];
std::array<type,size> array_name;
```

Static arrays are already built-in in C++ while dynamic arrays exploit the “pointer” definition of an array.

There are two types of arrays...

Static Arrays

- static == fixed
- Memory is allocated at compile time
- Placed in the stack memory.

```
type array_name[size];
std::array<type,size> array_name;
```

Dynamic Arrays

- dynamic == not fixed
- Memory is allocated whenever needed (needs to reallocate after)
- Placed in heap memory

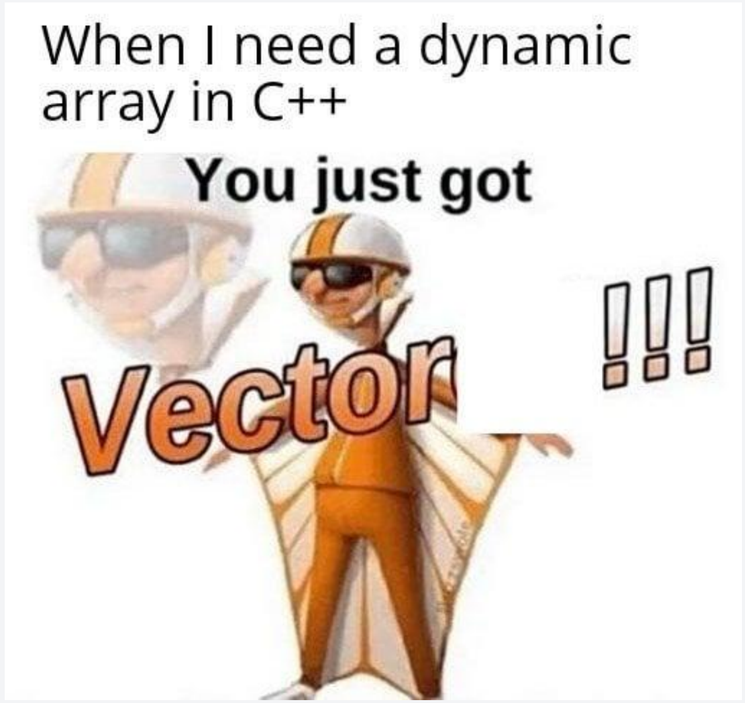
```
type* array_name = new type[size];
//need to call delete[] array_name after use
```

To dynamically increase the size of an array, you'd have to copy the initial elements, create a new array, and delete the old array. Too tedious!!!

Dynamic arrays using STL Vectors

STL Vector

- A dynamic array data structure in C++ standard library.
- Can be used by importing the `vector` library.



Dynamic arrays using STL Vectors

Initialization:

```
#include<vector>
using namespace std;
        …

vector<int> v = {1,2,3,4};
```

Dynamic arrays using STL Vectors

Initialization:

```
#include<vector>
using namespace std;
        …

vector<int> v = {1,2,3,4};
```

Methods:

```
cout << v[3] << “ “ << v.size() << end;      // output “4 4”
```

Dynamic arrays using STL Vectors

Initialization:

```
#include<vector>
using namespace std;
        …

vector<int> v = {1,2,3,4};
```

Methods:

```
cout << v[3] << “ “ << v.size() << end;     // output “4 4”
v.push_back(5);      // Adds it 5 at the end of the array
```

Dynamic arrays using STL Vectors

Initialization:

```
#include<vector>
using namespace std;
        …

vector<int> v = {1,2,3,4};
```

Methods:

```
cout << v[3] << “ “ << v.size() << end;     // output “4 4”
v.push_back(5);      // Adds it 5 at the end of the array
cout << v[4] << “ “ << v.size() << end;     // output “5 5”
```

Dynamic arrays using STL Vectors

Initialization:

```
#include<vector>
using namespace std;
        …

vector<int> v = {1,2,3,4};
```

Methods:

```
cout << v[3] << “ “ << v.size() << end;     // output “4 4”
v.push_back(5);      // Adds it 5 at the end of the array
cout << v[4] << “ “ << v.size() << end;     // output “5 5”
v.resize(10);        // contents are: 1 2 3 4 5 0 0 0 0 0
```

Dynamic arrays using STL Vectors

Initialization:

```
#include<vector>
using namespace std;
        …

vector<int> v = {1,2,3,4};
```

Methods:

```
cout << v[3] << “ “ << v.size() << end;     // output “4 4”
v.push_back(5);      // Adds it 5 at the end of the array
cout << v[4] << “ “ << v.size() << end;      // output “5 5”
v.resize(10);        // contents are: 1 2 3 4 5 0 0 0 0 0
v.clear();          // removes all elements
```

Dynamic arrays using STL Vectors

Initialization:

```
#include<vector>
using namespace std;
        …

vector<int> v = {1,2,3,4};
```

Methods:

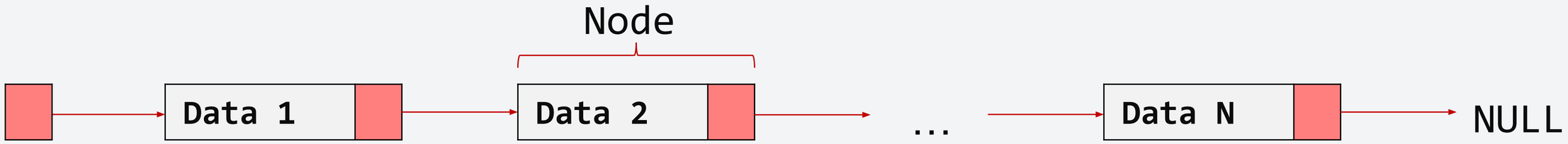
```
cout << v[3] << “ “ << v.size() << end;     // output “4 4”
v.push_back(5);      // Adds it 5 at the end of the array
cout << v[4] << “ “ << v.size() << end;     // output “5 5”
v.resize(10);        // contents are: 1 2 3 4 5 0 0 0 0 0
v.clear();          // removes all elements
cout << v.size() << endl;      //output “0”
```

Demo: Arrays and Vectors

LDS 2: Linked List

Linked list uses a component called **nodes**.

- A linked list is essentially a series of connected nodes.
- Each node contains an element.
- The connections between each node are made via pointers.



There are some libraries that implement linked list, but we’re not going to discuss. We’ll try to implement it as a new class.

Let’s try to create a Node class first

Implementation:

```
template<typename S>
class Node {
    public:
        S data;
        Node<S>* next;
        Node(S data, Node* next = NULL);
};
```

Let’s try to create a Node class first

Implementation:

```
template<typename S>
class Node {
    public:
        S data;
        Node<S>* next;
        Node(S data, Node* next = NULL);
};
```

Why did we use template?

Let’s try to create a Node class first

Implementation:

```
template<typename S>
class Node {
    public:
        S data;
        Node<S>* next;
        Node(S data, Node* next = NULL);
};
```

We used template so that the class can be reused to other data types

Let’s try to create a Node class first

Implementation:

```
template<typename S>
class Node {
    public:
        S data;
        Node<S>* next;
        Node(S data, Node* next = NULL);
};
```

The class has three members. Can you name them?

Let’s try to create a Node class first

Implementation:

```
template<typename S>
class Node {
    public:
        S data;
        Node<S>* next;
        Node(S data, Node* next = NULL);
};
```

The class has three members. Can you name them?

Let’s try to create a Node class first

Implementation:

```
template<typename S>
class Node {
    public:
        S data;
        Node<S>* next;
        Node(S data, Node* next = NULL);
};
```

Which is the constructor?

Let’s try to create a Node class first

Implementation:

```
template<typename S>
class Node {
    public:
        S data;
        Node<S>* next;
        Node(S data, Node* next = NULL);
};
```

Which is the constructor?

Let’s try to create a Node class first

Implementation:

```
template<typename S>
class Node {
    public:
        S data;
        Node<S>* next;
        Node(S data, Node* next = NULL);
};
```

The node class contains data as its value and next as a pointer to the next node.

Let’s see a Node in action!

Sample Usage:

```
template<typename S>
class Node { … };

template<typename S>
Node<S>::Node(S data, Node* next):data(data),next(next) {}
    …
Node<int> node1 = Node<int>(1, NULL);
Node<int> node2 = Node<int>(2);
Node<int> node3 = Node<int>(3); //same as Node(3,NULL)
```

Let’s see a Node in action!

Sample Usage:

```
template<typename S>
class Node { … };
```

Class definition

Constructor Definition

```
template<typename S>
Node<S>::Node(S data, Node* next):data(data),next(next) {}
```

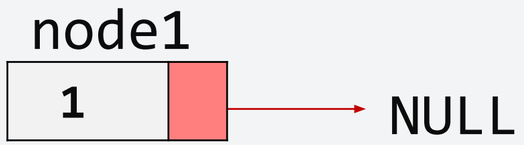
```
    …
Node<int> node1 = Node<int>(1, NULL);
Node<int> node2 = Node<int>(2);
Node<int> node3 = Node<int>(3); //same as Node(3,NULL)
```

Let’s check the output line-by-line!

Let’s see a Node in action!

Sample Usage:

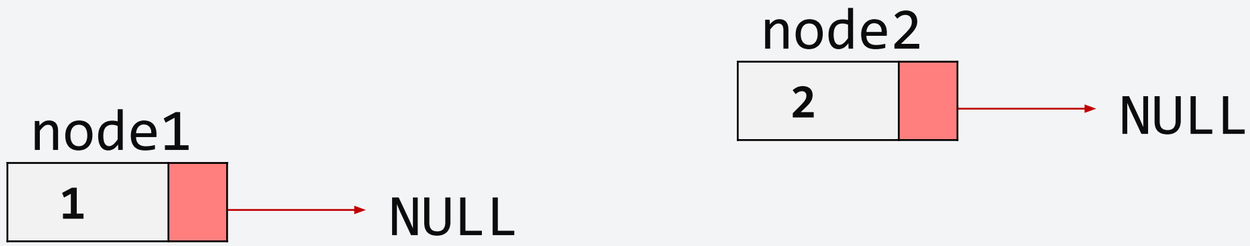
```
template<typename S>
class Node { … };

template<typename S>
Node<S>::Node(S data, Node* next):data(data),next(next) {}
    …
Node<int> node1 = Node<int>(1, NULL);
Node<int> node2 = Node<int>(2);
Node<int> node3 = Node<int>(3); //same as Node(3,NULL)
```



Let’s see a Node in action!

Sample Usage:

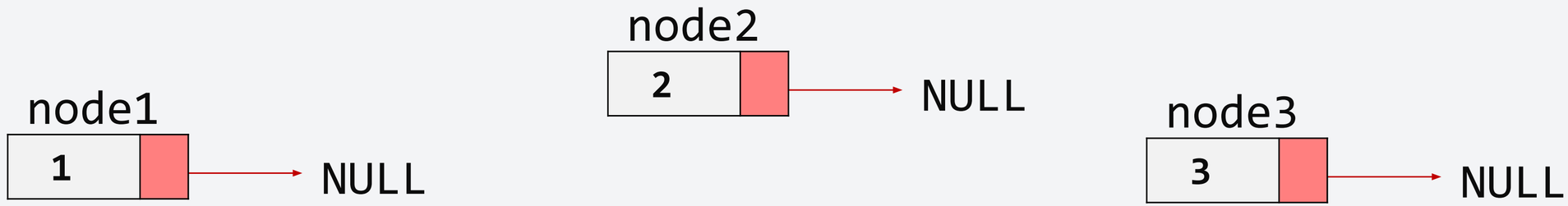
```
template<typename S>
class Node { … };

template<typename S>
Node<S>::Node(S data, Node* next):data(data),next(next) {}
    …
Node<int> node1 = Node<int>(1, NULL);
Node<int> node2 = Node<int>(2);
Node<int> node3 = Node<int>(3); //same as Node(3,NULL)
```



Let’s see a Node in action!

Sample Usage:

```
template<typename S>
class Node { … };

template<typename S>
Node<S>::Node(S data, Node* next):data(data),next(next) {}
    …
Node<int> node1 = Node<int>(1, NULL);
Node<int> node2 = Node<int>(2);
Node<int> node3 = Node<int>(3); //same as Node(3,NULL)
```



What if we update next?

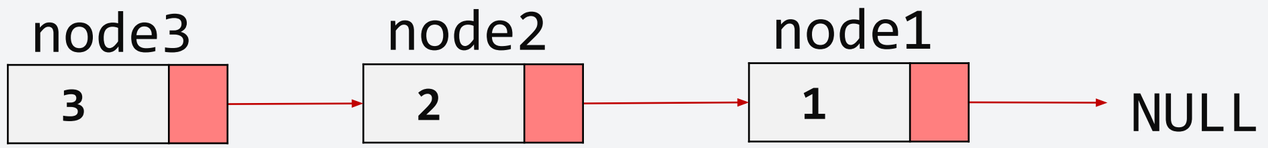
Sample Usage:

```
Node<int> node1 = Node<int>(1, NULL);
Node<int> node2 = Node<int>(2);
Node<int> node3 = Node<int>(3); //same as Node(3,NULL)

node3.next = &node2;
node2.next = &node1;
```

What if we update next?

Sample Usage:

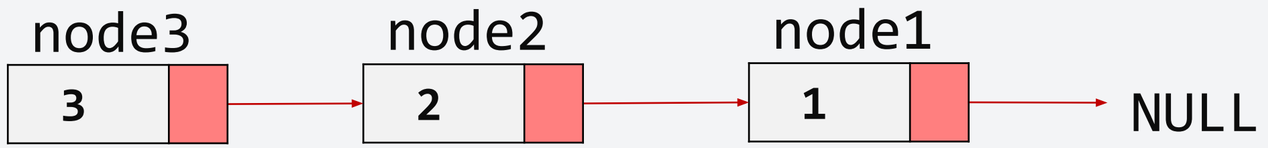
```
Node<int> node1 = Node<int>(1, NULL);
Node<int> node2 = Node<int>(2);
Node<int> node3 = Node<int>(3); //same as Node(3,NULL)

node3.next = &node2;
node2.next = &node1;
```



What if we update next?

Sample Usage:

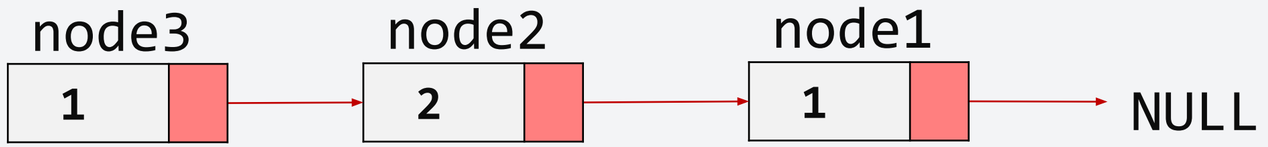
```
Node<int> node1 = Node<int>(1, NULL);
Node<int> node2 = Node<int>(2);
Node<int> node3 = Node<int>(3); //same as Node(3,NULL)

node3.next = &node2;
node2.next = &node1;
```



What if we update data?

Sample Usage:

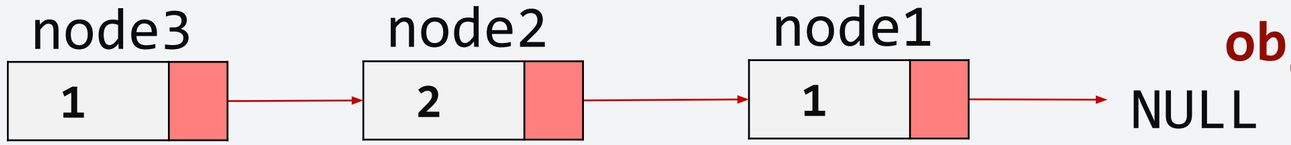
```
Node<int> node1 = Node<int>(1, NULL);
Node<int> node2 = Node<int>(2);
Node<int> node3 = Node<int>(3); //same as Node(3,NULL)

node3.next = &node2;
node2.next = &node1;

node3.data = 1;  //node3 data becomes 1
node2.next -> data = 3; //node1 becomes 3. WHY?
```



What if we update data?

Sample Usage:

```
Node<int> node1 = Node<int>(1, NULL);
Node<int> node2 = Node<int>(2);
Node<int> node3 = Node<int>(3); //same as Node(3,NULL)

node3.next = &node2;
node2.next = &node1;

node3.data = 1;  //node3 data becomes 1
node2.next -> data = 3; //node1 becomes 3. WHY?
```



Recall that we can access the data members of a class by using the notation:

`objname.datamember`

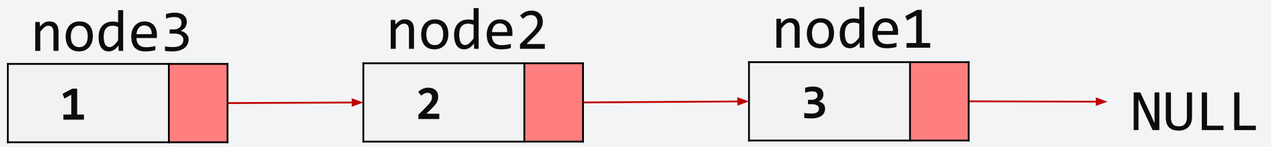
What if we update data?

Sample Usage:

```
Node<int> node1 = Node<int>(1, NULL);
Node<int> node2 = Node<int>(2);
Node<int> node3 = Node<int>(3); //same as Node(3,NULL)

node3.next = &node2;
node2.next = &node1;
```

```
node3.data = 1;  //node3 data becomes 1
node2.next -> data = 3; //node1 becomes 3. WHY?
```

Additionally, recall that we can access it by using -> similar to the ones here.

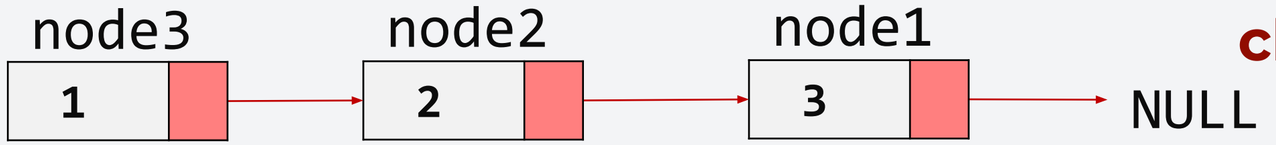


What if we update data?

Sample Usage:

```
Node<int> node1 = Node<int>(1, NULL);
Node<int> node2 = Node<int>(2);
Node<int> node3 = Node<int>(3); //same as Node(3,NULL)

node3.next = &node2;
node2.next = &node1;
```

```
node3.data = 1;  //node3 data becomes 1
node2.next -> data = 3; //node1 becomes 3. WHY?
```



Essentially, we are accessing the next node (since its a pointer) and changing its data.

Formalizing the Linked List Class

Class Implementation

Formalizing the Linked List Class

Class Implementation

```
template<typename S>
class Node { … };

template<typename S>
Node<S>::Node(S data, Node* next):data(data),next(next) {}

template<typename T>
class LinkedList {
    private:
        Node<T>* head;
        int size;
    public:
        LinkedList();
        void insertData(T val, int pos = 0);
        void deleteNode(int pos = 0);
        T getValue(int pos = 0);
        int getSize();
};
```

/** See Code 4.1 for implementation details **/

Formalizing the Linked List Class

Class Implementation

```
template<typename S>
class Node { … };

template<typename S>
Node<S>::Node(S data, Node* next):data(data),next(next) {}

template<typename T>
class LinkedList {
    private:
        Node<T>* head;
        int size;
    public:
        LinkedList();
        void insertData(T val, int pos = 0);
        void deleteNode(int pos = 0);
        T getValue(int pos = 0);
        int getSize();
};
```

Constructor that creates an empty linked list

Formalizing the Linked List Class

Class Implementation

```
template<typename S>
class Node { … };

template<typename S>
Node<S>::Node(S data, Node* next):data(data),next(next) {}

template<typename T>
class LinkedList {
    private:
        Node<T>* head;
        int size;
    public:
        LinkedList();
        void insertData(T val, int pos = 0);
        void deleteNode(int pos = 0);
        T getValue(int pos = 0);
        int getSize();
};
```

Class method that inserts a new element of the linked list into desired position

Formalizing the Linked List Class

Class Implementation

```
template<typename S>
class Node { … };

template<typename S>
Node<S>::Node(S data, Node* next):data(data),next(next) {}

template<typename T>
class LinkedList {
    private:
        Node<T>* head;
        int size;
    public:
        LinkedList();
        void insertData(T val, int pos = 0);
        void deleteNode(int pos = 0);
        T getValue(int pos = 0);
        int getSize();
};
```

Class method that deletes element at specified position (default at 0)

Formalizing the Linked List Class

Class Implementation

```
template<typename S>
class Node { … };

template<typename S>
Node<S>::Node(S data, Node* next):data(data),next(next) {}

template<typename T>
class LinkedList {
    private:
        Node<T>* head;
        int size;
    public:
        LinkedList();
        void insertData(T val, int pos = 0);
        void deleteNode(int pos = 0);
        T getValue(int pos = 0);
        int getSize();
};
```

Class method that gets the element value at specified position (default at 0)

Formalizing the Linked List Class

Class Implementation

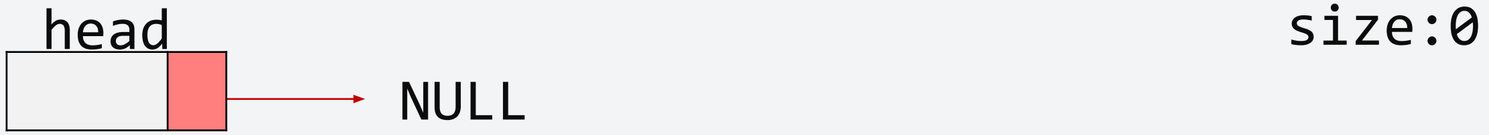
```
template<typename S>
class Node { … };

template<typename S>
Node<S>::Node(S data, Node* next):data(data),next(next) {}

template<typename T>
class LinkedList {
    private:
        Node<T>* head;
        int size;
    public:
        LinkedList();
        void insertData(T val, int pos = 0);
        void deleteNode(int pos = 0);
        T getValue(int pos = 0);
        int getSize();
};
```

Class method that returns the length of the linked list

Let’s try!

Sample Usage

```
template<typename S>
class Node { … };
   …
template<typename T>
class LinkedList { … };
          …

LinkedList<string> EEE121_handlers = LinkedList();
```

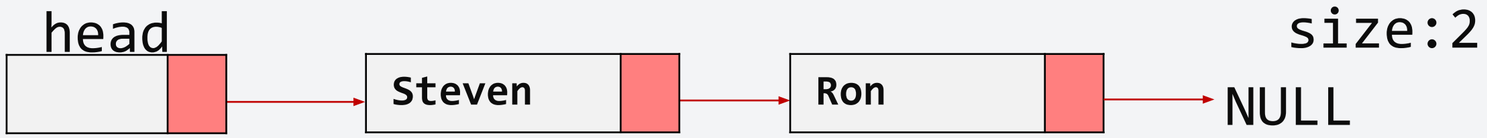


size:0

Let’s try!

Sample Usage

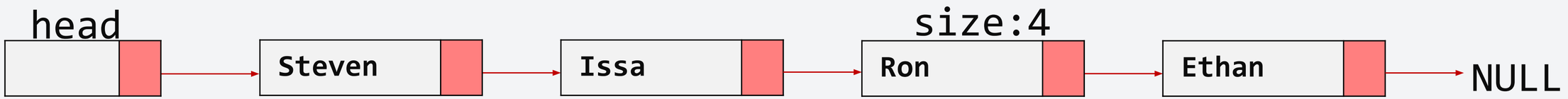
```
template<typename S>
class Node { … };
   …
template<typename T>
class LinkedList { … };
          …

LinkedList<string> EEE121_handlers = LinkedList();
EEE121_handlers.insertData(“Ron”);
EEE121_handlers.insertData(“Steven”);
```



Let’s try!

Sample Usage

```
template<typename S>
class Node { … };
   …
template<typename T>
class LinkedList { … };
          …

LinkedList<string> EEE121_handlers = LinkedList();
EEE121_handlers.insertData(“Ron”);
EEE121_handlers.insertData(“Steven”);
EEE121_handlers.insertData(“Ethan”,2);
EEE121_handlers.insertData(“Issa”,1);
```



Let’s try!

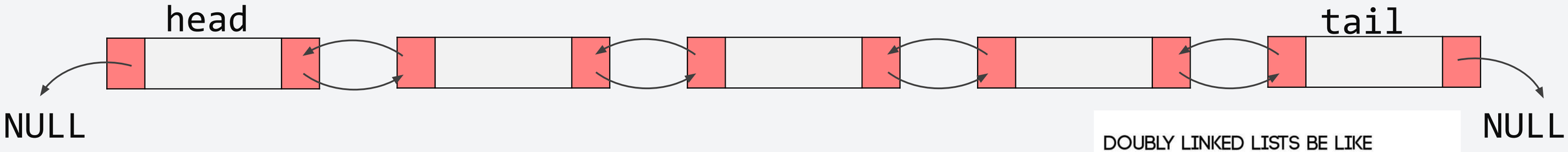
Sample Usage

```
template<typename S>
class Node { … };
   …
template<typename T>
class LinkedList { … };
          …

LinkedList<string> EEE121_handlers = LinkedList();
EEE121_handlers.insertData(“Ron”);
EEE121_handlers.insertData(“Steven”);
EEE121_handlers.insertData(“Ethan”,2);
EEE121_handlers.insertData(“Issa”,1);
EEE121_handlers.deleteNode(1);
```

Demo: Linked List

A variation, Doubly Linked List

- Accessing the last element of a linked list requires traversing all of the elements. Very time consuming right?
- Doubly linked list allows us to traverse a linked list in both directions.
  - We just need to add a new pointer!



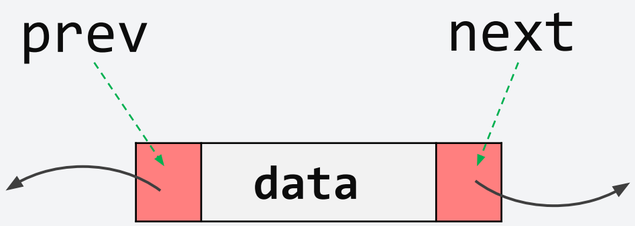
Changes:

- The node class now has two pointers: `next` and `prev`.
- Two pointers were initialized for `head` and `tail`.

A variation, Doubly Linked List

DNode class

```
template<typename S>
class DNode {
    public:
        S data;
        DNode<S>* next;
        Dnode<S>* prev;
  Node(S data, DNode* next = NULL, Dnode*
prev = NULL);
};
```



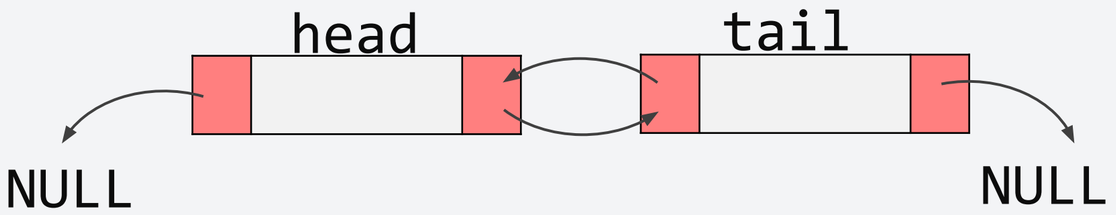
DLinkedList class

```
template<typename T>
class DLinkedList {
    private:
        DNode<T>* head;
        DNode<T>* tail;
        int size;
    public:
        DLinkedList();
        void addFront(T val);
        void addBack(T val);
        T removeFront();
        T removeBack();
        T front();
        T back();
        int getSize();
};
```

Let’s try!

Sample Usage

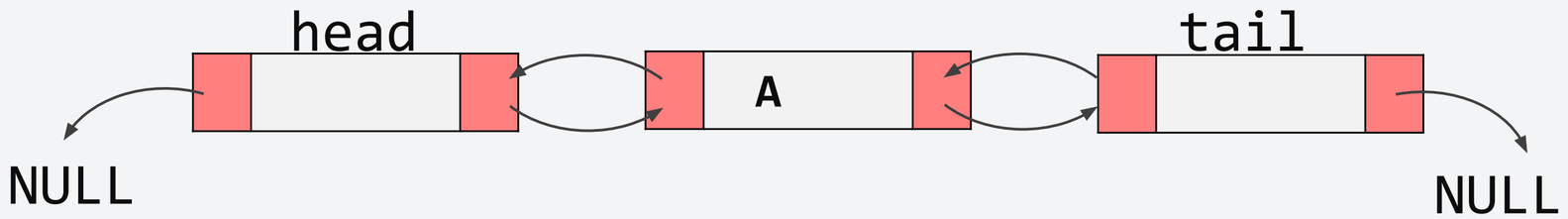
```
DLinkedList<string> letters = DLinkedList();
```



Let’s try!

Sample Usage

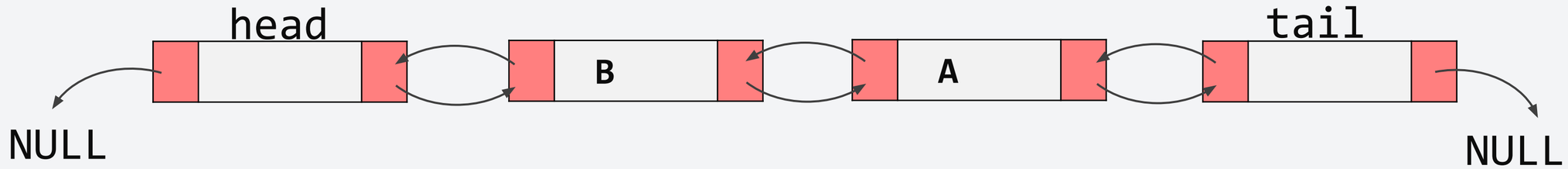
```
DLinkedList<string> letters = DLinkedList();
letters.addFront(“A”);
```



Let’s try!

Sample Usage

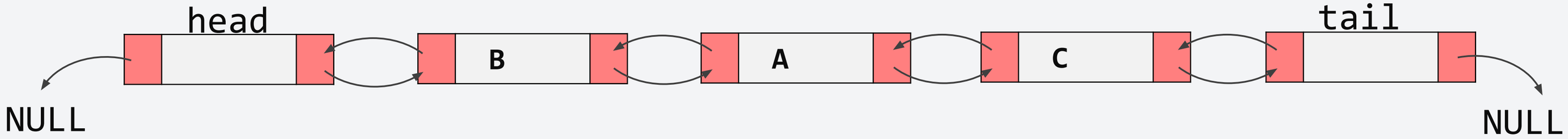
```
DLinkedList<string> letters = DLinkedList();
letters.addFront(“A”);
letters.addFront(“B”);
```



Let’s try!

Sample Usage

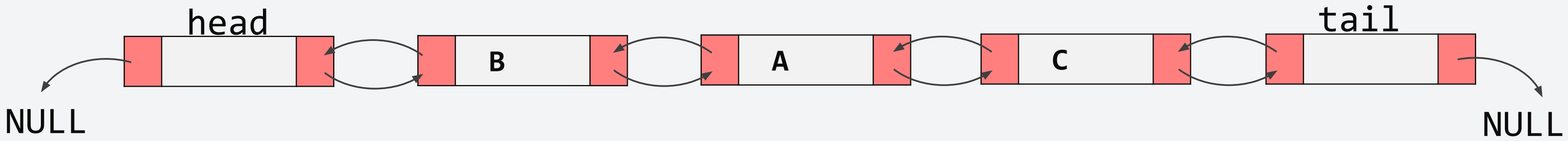
```
DLinkedList<string> letters = DLinkedList();
letters.addFront(“A”);
letters.addFront(“B”);
letters.addBack(“C”);
```



Let’s try!

Sample Usage

```
DLinkedList<string> letters = DLinkedList();
letters.addFront(“A”);
letters.addFront(“B”);
letters.addBack(“C”);

while(letters.getSize()) {
    cout << letters.removeBack() << “ “; // output “C A B \n”
}
cout << endl;
```



Which is better? Linked List or Doubly Linked List?

As repeatedly said before, there is no perfect algorithm and data structure. It all depends on the problem.

	Array	Singly Linked List	Doubly Linked List
Space Complexity	O(N)	O(N)	O(N)
Insert/Erase @ Front	O(N)	O(1)	O(1)
@ k-th entry	O(N)	O(N)	O(N)
@ Last	O(1)	O(N)	O(1)
Access the k-th entry	O(1)	O(N)	O(N)

Which is better? Linked List or Doubly Linked List?

As repeatedly said before, there is no perfect algorithm and data structure. It all depends on the problem.

	Array	Singly Linked List	Doubly Linked List
Space Complexity	O(N)	O(N)	O(N)
Insert/Erase @ Front	O(N)	O(1)	O(1)
@ k-th entry	O(N)	O(N)	O(N)
@ Last	O(1)	O(N)	O(1)
Access the k-th entry	O(1)	O(N)	O(N)

All of them have the same space complexity

Which is better? Linked List or Doubly Linked List?

As repeatedly said before, there is no perfect algorithm and data structure. It all depends on the problem.

	Array	Singly Linked List	Doubly Linked List
Space Complexity	O(N)	O(N)	O(N)
Insert/Erase @ Front	O(N)	O(1)	O(1)
@ k-th entry	O(N)	O(N)	O(N)
@ Last	O(1)	O(N)	O(1)
Access the k-th entry	O(1)	O(N)	O(N)

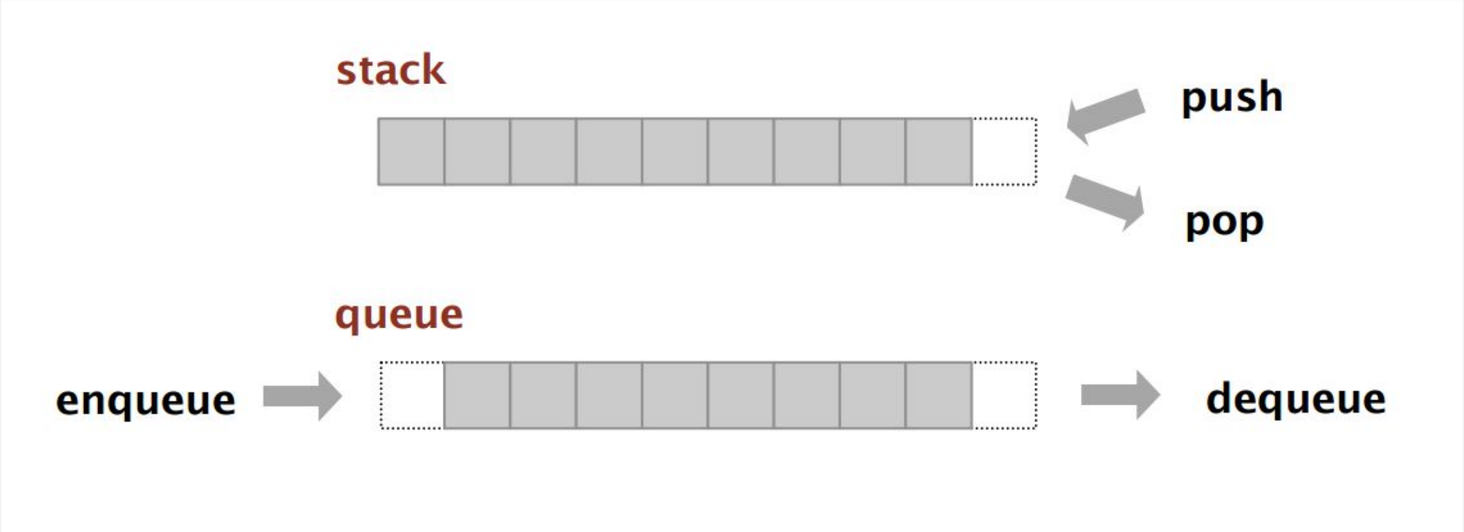
Linked lists for dynamic storage, array for fixed structure due to faster access

- Data Structures: An Overview
- Arrays and Linked Lists
- **Stacks and Queues**

Stacks and Queues

Stacks and Queues are data structures that are derived from arrays and linked lists:

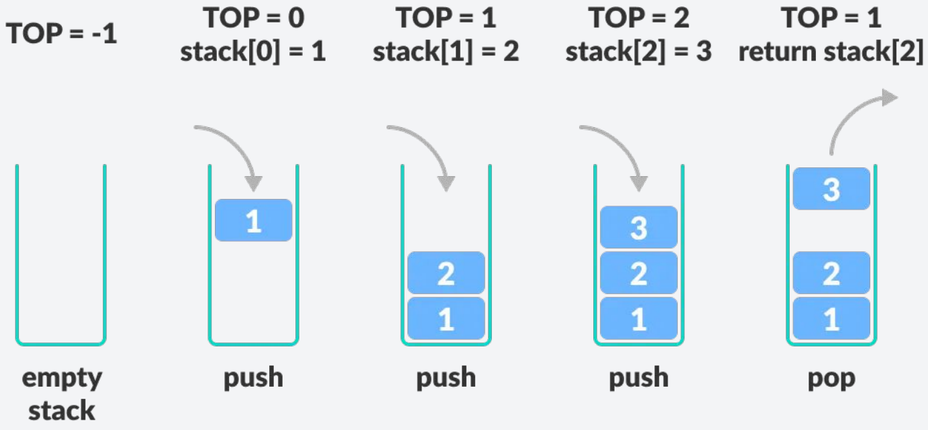
- **Stack**: container of objects that are inserted and removed according to the **last-in-first-out (LIFO)** principle
- **Queue**: container of objects that are inserted and removed according to the **first-in-first-out (FIFO)** principle



There are two types of arrays...

Stack

- stack.push(val) - adds elements at the end
- stack.pop - removes elements at the end



Queue

- queue.enqueue(val) - adds elements at the end
- queue.dequeue - removes elements at the front

Implementing stack and queue with Linked Lists

Using a Doubly linked list, we can implement a stack and queue.

Stack:

- `stack.push(val) == DLinkedList.addFront(val)`
- `stack.pop() == DLinkedList.removeFront()`

Queue:

- `queue.enqueue(val) == DLinkedList.addFront(val)`
- `queue.dequeue() == DLinkedList.removeBack()`

Recall: DLinkedList Methods

- addFront() - adds elements at the front
- removeFront - removes the front and return its data
- removeBack - removes the last node and returns its data

Implementing stack and queue with Arrays

Enqueue/Push	`top = (top == CAPACITY-1 ? 0:top+1) // Adds at the end of the array` `container[top] = val; // If already full, wrap around`

Pop	`val = container[top]; // Gets the value of the last`
	`top = (top == 0? CAPACITY-1:top-1) // Reduces top by 1`
	`return val; // If already at the end, wrap around`

Dequeue	`val = container[bottom]; // Gets the value of the first`
	`bottom = (bottom == CAPACITY-1? 0:bottom+1) // Reduces bottom by 1`
	`return val; // If already at the end, wrap around`

getSize	`return (top >= bottom? top-bottom:CAPACITY-(bottom-top));`

Built-in Libraries

There are already libraries that employ these non-primitive data structures. Our goal in this lesson was to make you understand the concept. So, you would have to familiarize yourself with the libraries by reading it in your own time :)

Stack

- **Declaration**: `std::array<type,size> array_name`
- **Common methods**: `size(), front(), back(), operator[], fill(), swap()`

Queue

- **Declaration**: `std::vector<type> vector_name`
- **Common methods**: `size(), empty(), front(), back(), push_back(), pop_back(), operator[], fill(), swap(), erase(), clear(), resize()`

Built-in Libraries

There are already libraries that employ these non-primitive data structures. Our goal in this lesson was to make you understand the concept. So, you would have to familiarize yourself with the libraries by reading it in your own time :)

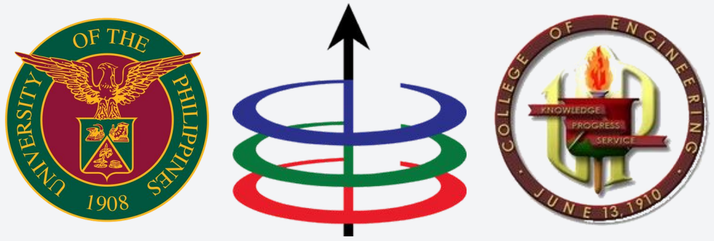
Singly Linked List

- **Declaration**: `std::forward_list<type> list_name`
- **Common methods**: `front()`, `push_front()`, `pop_front()`, `insert_after()`, `erase_after()`, `empty()`, `sort()`, `size()`, `clear()`

Doubly Linked List

- **Declaration**: `std::list<type> list_name`
- **Common methods**: `front()`, `back()`, `push_front()`, `pop_front()`, `push_back()`, `pop_back()`, `insert()`, `erase()`, `empty()`, `sort()`, `size()`, `clear()`

Any questions?



Lecture 4
Linear Data Structures

EEE 121 2s2526

Steven S. Sison